

Laboratório OAI (Core + gNB + UERANSIM)

Laboratório 5G usando **OpenAirInterface (OAI)** com três componentes separados para testes de failover e validação end-to-end, no contexto do projeto RNP Failover / CERISE.

Vector Packet Processing (VPP)

Este laboratório utiliza **Vector Packet Processing (VPP)** no plano de usuário. O Core é iniciado com a stack **basic-vpp**, que emprega o **UPF-VPP** (User Plane Function baseada em FD.io VPP) em vez do SPGWU-Tiny. O VPP oferece:

- **Alto desempenho:** processamento vetorial de pacotes em batch
- **Throughput elevado:** otimizado para tráfego GTP-U (N3/N9)
- **Arquitetura moderna:** baseada no framework FD.io VPP

Comando para subir o Core com VPP:

```
cd oai-cn5g-fed/docker-compose
python3 core-network.py --type start-basic-vpp --scenario 1
```

O projeto utiliza **três componentes** distintos:

- **Core OAI:** `oai-cn5g-fed/docker-compose/` — NFs do 5G Core (AMF, SMF, NRF, **UPF-VPP**, etc.)
- **gNB OAI:** `openairinterface5g/` — gNB e UE nativos (RFSIM), build a partir do código-fonte
- **UERANSIM:** `ueransim/docker-compose.yaml` — alternativa containerizada (gNB + UE) para testes e2e

Índice

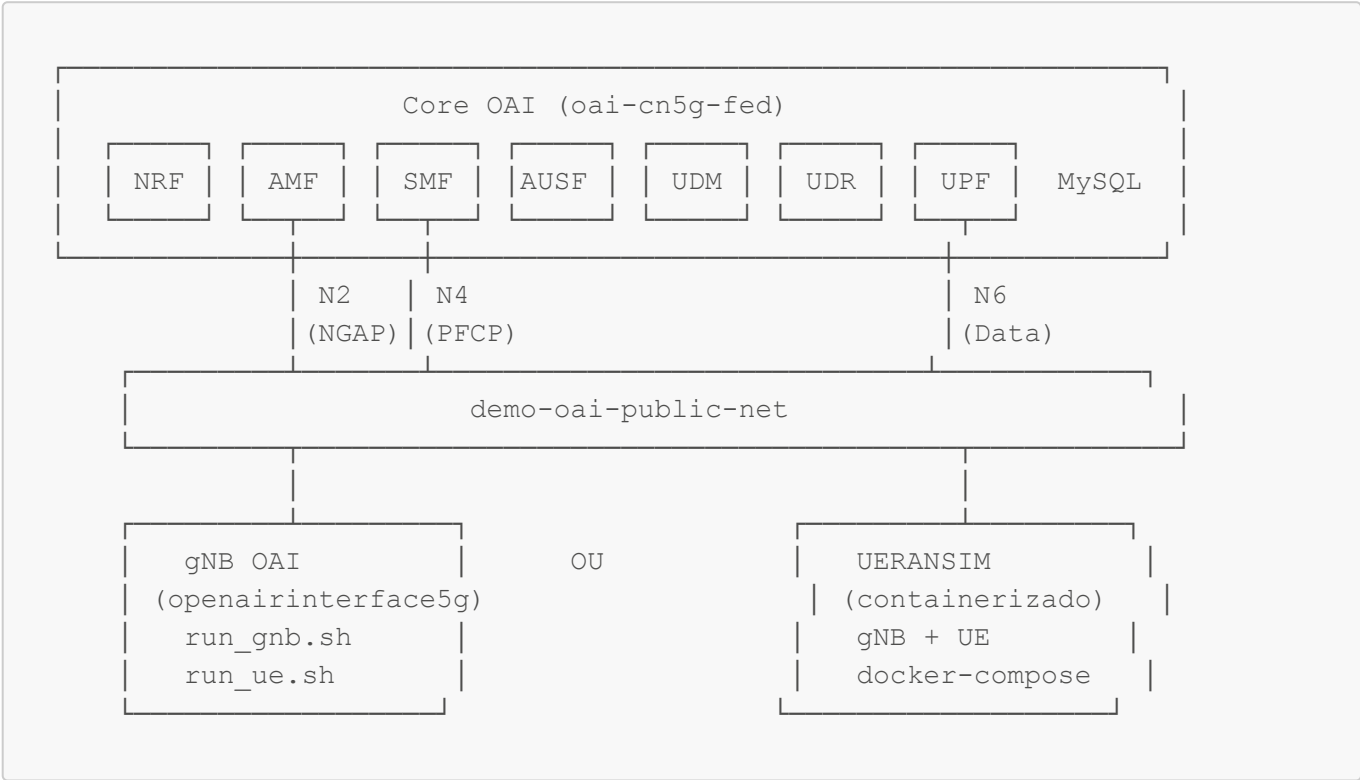
1. [Vector Packet Processing \(VPP\)](#)
2. [Pré-requisitos](#)
3. [Arquitetura](#)
4. [Estrutura de Diretórios](#)
5. [Início Rápido](#)
6. [Cenários de Deploy](#)
7. [Scripts e Comandos](#)
8. [Testes End-to-End](#)
9. [Troubleshooting](#)
10. [Guia de Reexecução Completo](#)

Pré-requisitos

- Docker 20.10+ e Docker Compose 2.0+
- Conta no Docker Hub (para pull das imagens OAI)

- Ubuntu 22.04+ (recomendado)
- IPv4 forwarding habilitado: `sudo sysctl net.ipv4.conf.all.forwarding=1` e `sudo iptables -P FORWARD ACCEPT`
- Python3 (para `core-network.py`)
- ~8GB RAM livre
- Para gNB OAI nativo: dependências de build e ~10 GB de disco (ver [docs/INSTALACAO_GNB_OAI.md](#))

Arquitetura



Estrutura dos Componentes

Componente	Localização	Tipo	Descrição
Core OAI	<code>oai-cn5g-fed/docker-compose/</code>	Docker Compose	NRF, AMF, SMF, AUSF, UDM, UDR, UPF-VPP (Vector Packet Processing), MySQL, DN
gNB OAI	<code>openairinterface5g/</code>	Binários nativos	gNB e nrUE em modo RFSIM, build a partir do código-fonte
UERANSIM	<code>ueransim/</code> ou <code>oai-cn5g-fed/docker-compose/docker-compose-ueransim-vpp.yaml</code>	Docker Compose	gNB + UE em um container, alternativa ao gNB OAI

Redes Docker (Core OAI)

- **demo-oai-public-net:** Rede principal para comunicação entre Core e RAN
- **oai-public-access:** Rede de acesso (GTP-U) entre gNB e UPF (quando usado UERANSIM com VPP)

Estrutura de Diretórios

```

oai-cn-gnb/
├── scripts/
│   ├── up_all.sh           # Sobe tudo (Core + UERANSIM + gNB OAI)
│   ├── down_all.sh        # Para tudo
│   ├── up_core.sh         # Iniciar Core OAI
│   ├── down_core.sh       # Parar Core OAI
│   ├── up_ueransim.sh     # Iniciar RAN UERANSIM (container)
│   ├── down_ueransim.sh   # Parar RAN UERANSIM
│   ├── up_gnb_oai.sh      # Iniciar RAN gNB OAI (nativo)
│   ├── down_gnb_oai.sh    # Parar RAN gNB OAI
│   ├── test-vpp-throughput.sh # Teste de throughput (iperf3)
│   └── fix-ue-subscriber.sh # Adiciona usuários ao DB (apenas se DB
antigo)
├── oai-cn5g-fed/          # Core OAI
│   ├── docker-compose/
│   │   ├── core-network.py      # Script de deploy (start/stop)
│   │   ├── docker-compose-basic-nrf.yaml
│   │   ├── docker-compose-basic-vpp-nrf.yaml
│   │   ├── docker-compose-ueransim-vpp.yaml # UERANSIM + Core VPP
│   │   └── docker-compose-no-privilege.yaml
│   │   └── database/
│   └── docs/
│       ├── DEPLOY_SA5G_BASIC_DEPLOYMENT.md
│       ├── DEPLOY_SA5G_WITH_UERANSIM.md
│       └── DEPLOY_SA5G_WITH_VPP_UPF.md
├── openairinterface5g/    # gNB OAI (RAN nativo)
│   ├── cmake_targets/     # Build output
│   ├── scripts/
│   │   ├── run_gnb.sh
│   │   ├── run_ue.sh
│   │   └── gnb.conf
│   └── ...
├── ueransim/              # RAN containerizado (alternativa)
│   ├── configs/
│   │   ├── gnb.yaml
│   │   ├── ue.yaml
│   │   └── entrypoint.sh
│   └── docker-compose.yaml
└── README.md              # Este arquivo

```

Início Rápido

Cenário 1: Core OAI + UERANSIM (recomendado para testes e2e)

Passo 1 — Pull das imagens e sync do repositório (uma vez):

```
cd code/oai-cn-gnb
docker login
# Pull das imagens OAI (ver seção "Pull de Imagens")
```

Passo 2 — Iniciar o Core:

```
./scripts/up_core.sh
```

Passo 3 — Iniciar UERANSIM:

```
./scripts/up_ueransim.sh
```

Passo 4 — Verificar conectividade:

```
docker exec ueransim ping -c 3 -I uesimtun0 google.com
```

Parar o laboratório:

```
./scripts/down_ueransim.sh
./scripts/down_core.sh
```

Cenário 2: Core OAI + gNB OAI (RFSIM)

Passo 1 — Iniciar o Core (como acima).

Passo 2 — Build do openairinterface5g (uma vez):

```
cd openairinterface5g/cmake_targets
./build_oai --ninja -I
./build_oai --ninja --gNB --nrUE -w SIMU -c
```

Passo 3 — Configurar rede (após o Core estar rodando):

```
sudo ip addr add 192.168.70.129/24 dev demo-oai 2>/dev/null || true
```

Passo 4 — Iniciar gNB OAI:

```
./scripts/up_gnb_oai.sh
```

Parar:

```
./scripts/down_gnb_oai.sh
./scripts/down_core.sh
```

Guia completo: [docs/INSTALACAO_GNB_OAI.md](#)

Cenários de Deploy

Scripts (recomendado)

Script	Descrição
<code>./scripts/up_all.sh</code>	Sobe tudo (Core + UERANSIM + gNB OAI)
<code>./scripts/down_all.sh</code>	Para tudo (ordem inversa)
<code>./scripts/up_core.sh</code>	Inicia Core OAI (UPF-VPP, NRF)
<code>./scripts/down_core.sh</code>	Para o Core
<code>./scripts/up_ueransim.sh</code>	Inicia RAN UERANSIM (gNB + UE em container)
<code>./scripts/down_ueransim.sh</code>	Para UERANSIM
<code>./scripts/up_gnb_oai.sh</code>	Inicia RAN gNB OAI (gNB + nrUE nativos)
<code>./scripts/down_gnb_oai.sh</code>	Para gNB OAI
<code>./scripts/test-vpp-throughput.sh</code>	Teste de throughput (iperf3, UERANSIM ou nrUE)

Core OAI (comandos diretos)

Comando	Descrição
<code>python3 core-network.py --type start-basic-vpp --scenario 1</code>	Inicia Core com UPF-VPP e NRF
<code>python3 core-network.py --type start-basic --scenario 1</code>	Inicia Core com SPGWU-Tiny e NRF
<code>python3 core-network.py --type stop-basic-vpp --scenario 1</code>	Para o Core

UERANSIM

Comando	Descrição
<code>./scripts/up_ueransim.sh</code>	Inicia UERANSIM (usa redes do Core)

Comando	Descrição
<code>./scripts/down_ueransim.sh</code>	Para UERANSIM
<code>docker compose -f docker-compose-ueransim-vpp.yaml up -d</code>	Comando direto (no dir docker-compose)

gNB OAI (nativo)

Comando	Descrição
<code>./scripts/up_gnb_oai.sh</code>	Inicia gNB + nrUE (RFSIM) em background
<code>./scripts/down_gnb_oai.sh</code>	Para gNB e nrUE
<code>./run_gnb.sh / ./run_ue.sh</code>	Comandos manuais (executar de <code>openairinterface5g/scripts/</code>)

Documentação detalhada: docs/INSTALACAO_GNB_OAI.md — instalação, build e troubleshooting.

Scripts e Comandos

Pull de Imagens OAI

```
docker pull oaisoftwarealliance/oai-amf:v1.5.1
docker pull oaisoftwarealliance/oai-nrf:v1.5.1
docker pull oaisoftwarealliance/oai-spgwu-tiny:v1.5.1
docker pull oaisoftwarealliance/oai-smf:v1.5.1
docker pull oaisoftwarealliance/oai-udr:v1.5.1
docker pull oaisoftwarealliance/oai-udm:v1.5.1
docker pull oaisoftwarealliance/oai-ausf:v1.5.1
docker pull oaisoftwarealliance/oai-upf-vpp:v1.5.1
docker pull oaisoftwarealliance/oai-nssf:v1.5.1
docker pull oaisoftwarealliance/oai-pcf:v1.5.1
docker pull oaisoftwarealliance/oai-nef:v1.5.1
docker pull oaisoftwarealliance/trf-gen-cn5g:latest
```

Sync do repositório OAI CN5G

```
git clone --branch v1.5.1 https://gitlab.eurecom.fr/oai/cn5g/oai-cn5g-fed.git
cd oai-cn5g-fed
git checkout -f v1.5.1
./scripts/syncComponents.sh
git submodule deinit --all --force
git submodule init
git submodule update
```

Configuração AMF para UERANSIM

UERANSIM não suporta NIA0/NEA0. O `docker-compose-basic-vpp-nrf.yaml` já inclui:

```
- INT_ALGO_LIST=["NIA1" , "NIA2"]  
- CIPH_ALGO_LIST=["NEA1" , "NEA2"]
```

Diagnóstico e correção de conexão UE

Se o UE ficar em `SGMM-REG-INITIATED`:

```
./scripts/diagnose-ue-connection.sh    # Diagnóstico  
./scripts/fix-ue-subscriber.sh         # Adiciona subscriber ao banco (se  
necessário)
```

Depois reinicie Core e UERANSIM.

Testes End-to-End

Com UERANSIM

1. **Registro do UE:** Verificar logs `docker logs ueransim` para "MM-REGISTERED"
2. **Sessão PDU:** Verificar interface `uesimtun0` no container
3. **Conectividade:** `docker exec ueransim ping -c 3 -I uesimtun0 8.8.8.8`

Com gNB OAI

1. **NG Setup:** Verificar logs do gNB para "NG Setup procedure is successful"
2. **Registro UE:** Verificar logs do nrUE
3. **Tráfego:** Usar `trf-gen-cn5g` ou ferramentas de teste

Teste de Throughput VPP (iperf3)

Mede o throughput do plano de usuário através do UPF-VPP. O script usa `-B` para forçar o tráfego pela interface do túnel (`uesimtun0` ou `oaitun_ue*`) e inclui timeout para evitar travamento.

```
./scripts/test-vpp-throughput.sh
```

Variáveis opcionais:

- `UE_SOURCE=nrue` — usar nrUE (OAI) em vez de UERANSIM
- `IPERF_DURATION=15` — duração em segundos (padrão: 10)
- `IPERF_MODE=udp` — modo UDP em vez de TCP
- `OAI_DN_IP=192.168.73.135` — IP do DN (se diferente)

Testes de Failover

Para cenários de failover de UPF, consulte a documentação do projeto principal em [rnp_failover/README.md](#). O laboratório OAI pode ser integrado com múltiplas UPFs conforme a configuração do `docker-compose` e do SMF.

Troubleshooting

PCF/UDR ou outras NFs reiniciando

- Verificar se MySQL está healthy: `docker ps | grep mysql`
- Verificar logs: `docker logs oai-amf, docker logs oai-smf`
- Verificar conectividade entre containers na rede `demo-oai-public-net`

`run_gnb.sh`: "No such file or directory" ou "nr-softmodem: command not found"

- **Causa:** Script executado do diretório errado ou build não feito.
- **Solução:** Use `./scripts/up_gnb_oai.sh` (recomendado) ou execute `run_gnb.sh` de `openairinterface5g/scripts/`. Para build: `cd openairinterface5g/cmake_targets && ./build_oai --ninja --gNB --nrUE -w SIMU -c`. Ver [docs/INSTALACAO_GNB_OAI.md](#).

gNB OAI: "NG setup failure" / AMF não loga o gNB

- **Causa:** S-NSSAI do gNB não coincide com o AMF. O AMF usa SST=222, SD=123.
- **Solução:** Em `openairinterface5g/scripts/gnb.conf`, use `plmn_list = ({ ... snssaiList = ({ sst = 222; sd = 123; }) })`. Ver [docs/INSTALACAO_GNB_OAI.md](#).

gNB não conecta ao AMF

- Verificar TAC, PLMN (MCC/MNC) entre gNB e AMF
- Verificar rota do host gNB para o host do Core
- Para gNB OAI nativo: adicionar IP ao host (`sudo ip addr add 192.168.70.129/24 dev demo-oai`) e verificar `ping 192.168.70.132`
- Para UERANSIM: verificar `NGAP_PEER_IP` e `LINK_IP` no `docker-compose`

UE não acessa internet

- Verificar se UPF está healthy
- Verificar logs do SMF: `docker logs oai-smf | grep PFCP`
- Verificar rota no UE: `docker exec ueransim ip route` (UERANSIM)

`docker-compose-no-privilege`

Se usar `docker-compose-no-privilege.yaml`, comente as linhas `cap_drop: - ALL` antes de subir os serviços.

Guia de Reexecução Completo

Roteiro passo a passo para reproduzir o laboratório do zero. Execute na ordem indicada.

Fase 1: Setup Inicial (uma vez)

```
cd oai-cn-gnb

# 1. Pull das imagens OAI
docker login
docker pull oaisoftwarealliance/oai-amf:v1.5.1
docker pull oaisoftwarealliance/oai-nrf:v1.5.1
docker pull oaisoftwarealliance/oai-smf:v1.5.1
docker pull oaisoftwarealliance/oai-udr:v1.5.1
docker pull oaisoftwarealliance/oai-udm:v1.5.1
docker pull oaisoftwarealliance/oai-ausf:v1.5.1
docker pull oaisoftwarealliance/oai-upf-vpp:v1.5.1
docker pull oaisoftwarealliance/trf-gen-cn5g:latest

# 2. Build do gNB OAI (RFSIM) - ~15-30 min
cd openairinterface5g/cmake_targets
./build_oai --ninja -I
./build_oai --ninja --gNB --nrUE -w SIMU -c
cd ../../
```

Fase 2: Usuários pré-cadastrados

Os **usuários finais 01 e 02** já estão em `oai_db2.sql`:

Usuário	IMSI	Uso
01	208950000000031	UERANSIM (docker-compose-ueransim-vpp)
02	208950000000032	nrUE (openairinterface5g/scripts/ue.conf)

Importante: Se o banco foi criado antes desta alteração, recrie o volume: `docker compose down -v` (no dir do Core) e suba novamente. Ou use `./scripts/fix-ue-subscriber.sh`.

Fase 3: Configurações Críticas

Arquivo	Parâmetro	Valor	Motivo
<code>openairinterface5g/scripts/gnb.conf</code>	<code>plmn_list</code>	<code>sst = 222; sd = 123</code>	Deve coincidir com AMF (SST_0, SD_0)
<code>openairinterface5g/scripts/ue.conf</code>	<code>imsi</code>	<code>208950000000032</code>	Usuário 02 (nrUE)
<code>openairinterface5g/scripts/ue.conf</code>	<code>nssai_sst, nssai_sd</code>	<code>222, 123</code>	Slice do AMF
<code>openairinterface5g/scripts/ue.conf</code>	<code>dnn</code>	<code>default</code>	DNN do SMF para slice 222/123

Arquivo	Parâmetro	Valor	Motivo
ueransim/configs/gnb.yaml	slices	sst: 222, sd: "000123"	Alinhado ao AMF
docker-compose-basic-vpp-nrf.yaml	AMF	INT_ALGO_LIST, CIPH_ALGO_LIST	UERANSIM não suporta NIA0/NEA0

Fase 4: Deploy (ordem de execução)

Opção única — sobe tudo:

```
cd oai-cn-gnb
./scripts/up_all.sh
```

Ou passo a passo:

```
cd oai-cn-gnb

# 1. Iniciar Core OAI (UPF-VPP)
./scripts/up_core.sh

# 2. Configurar rede para gNB OAI (interface demo-oai)
sudo ip addr add 192.168.70.129/24 dev demo-oai 2>/dev/null || true

# 3. Iniciar UERANSIM (usuário 01)
./scripts/up_ueransim.sh

# 4. Iniciar gNB OAI (usuário 02)
./scripts/up_gnb_oai.sh
```

Fase 5: Verificação

```
# Core e containers
docker ps -a

# UERANSIM: UE registrado e conectividade
docker exec ueransim ping -c 3 -I uesimtun0 8.8.8.8

# gNB OAI: interface oaitun no host
ip addr show oaitun_ue0 # ou oaitun_ue1

# AMF: ambos os gNBs conectados (ver logs)
docker logs oai-amf 2>&1 | grep -E "gNB|Connected"

# Deve mostrar: UERANSIM-gnb e OAI-gNB
```

Fase 6: Teste de Throughput

O script usa `-B` para forçar o tráfego pela interface do túnel (uesimtun0 ou oaitun_ue*).

```
# Com UERANSIM (padrão)
./scripts/test-vpp-throughput.sh

# Com nrUE (OAI)
UE_SOURCE=nrue ./scripts/test-vpp-throughput.sh

# Opções adicionais
IPERF_DURATION=15 IPERF_MODE=udp ./scripts/test-vpp-throughput.sh
```

Teste manual (iperf3):

```
# Servidor no DN (terminal 1)
docker exec -it oai-ext-dn iperf3 -s

# Cliente UERANSIM (terminal 2)
docker exec ueransim iperf3 -c 192.168.73.135 -t 10 -f m -B 12.1.1.2

# Cliente nrUE (terminal 2, no host)
iperf3 -c 192.168.73.135 -t 10 -f m -B $(ip -4 addr show oaitun_ue1 | grep
-oP 'inet \K[\d.]+' )
```

Fase 7: Parada (ordem inversa)

```
./scripts/down_all.sh
```

Ou passo a passo: `down_gnb_oai.sh` → `down_ueransim.sh` → `down_core.sh`

Resumo de IPs e Redes

Elemento	IP/Rede	Interface
AMF	192.168.70.132	demo-oai-public-net
gNB OAI (host)	192.168.70.129	demo-oai
UERANSIM	192.168.70.141	demo-oai-public-net
DN (oai-ext-dn)	192.168.73.135	cn5g-core
UE (túnel)	12.1.1.x	uesimtun0 / oaitun_ue*

Documentação Adicional

- **gNB OAI (instalação e uso):** [docs/INSTALACAO_GNB_OAI.md](#)
 - **Core OAI:** [oai-cn5g-fed/docs/DEPLOY_SA5G_BASIC_DEPLOYMENT.md](#)
 - **UERANSIM com OAI:** [oai-cn5g-fed/docs/DEPLOY_SA5G_WITH_UERANSIM.md](#)
 - **UPF-VPP (Vector Packet Processing):** [oai-cn5g-fed/docs/DEPLOY_SA5G_WITH_VPP_UPF.md](#)
— documentação detalhada do VPP no OAI
 - **Projeto principal:** [rnp_failover/README.md](#)
-

Relação com o Projeto RNP Failover

Este laboratório faz parte do repositório **RNP Failover** (CERISE/UFG), que investiga:

- Escalabilidade e tolerância a falhas do 5G Core
- Failover de UPF e otimização de performance
- Testes end-to-end e validação de cenários

O OAI oferece uma stack alternativa ao Open5GS e free5GC para comparação e testes. A estrutura com Core, gNB e UERANSIM separados permite flexibilidade para cenários de pesquisa.

Última Atualização: 2026-02-26